

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 7

Contador binario ascendente
de 4 bits mediante pulsador

Resistencia pull-down, anti-rebote y representación binaria

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: 25 de febrero de 2026

“Contar en binario es entender el lenguaje nativo de las máquinas”

Índice

1. Introducción	2
1.1. Contexto y Justificación	2
1.2. Objetivos	2
2. Marco Teórico	2
2.1. Sistema Binario y Representación	2
2.2. Función gpio_set_pulls()	2
2.3. Operaciones de Bits utilizadas	3
3. Desarrollo	3
3.1. CMakeLists.txt	3
3.2. Practica_7.c	3
4. Diagramas del Circuito	5
4.1. Diagrama Esquemático	5
4.2. Diagrama en Protoboard	5
4.3. Conexiones	5
5. Preguntas de Repaso	6
5.1. 1. ¿Qué pasa si gpio_set_pulls(BUTTON, false, true) cambia a gpio_set_pulls(BUTTON, true, false)?	6
5.2. 2. ¿Cuál es la función del cable rojo que sale del pin 40 al riel positivo de la protoboard?	6
5.3. 3. Explica el bloque de código de la línea 14 a 19	6
6. Conclusiones	6
6.1. Posibles Mejoras	6
7. Referencias	7

1. Introducción

1.1. Contexto y Justificación

La representación binaria es fundamental en los sistemas digitales. Un contador binario de 4 bits permite visualizar físicamente cómo los microcontroladores manejan los números en su interior, utilizando LEDs para representar cada bit.

En esta práctica se implementa un contador ascendente de 0 a 15 (0000 a 1111 binario) controlado por un pulsador. Cada vez que se presiona el botón, el contador se incrementa en 1 y los 4 LEDs muestran el valor actual.

1.2. Objetivos

- Utilizar la resistencia pull-down interna del RP2040 mediante `gpio_set_pulls()`
- Implementar un sistema anti-rebote para evitar lecturas falsas del pulsador
- Controlar 4 LEDs como salida para representar un número binario de 4 bits
- Aprender a manipular bits (desplazamientos y máscaras) para actualizar LEDs

2. Marco Teórico

2.1. Sistema Binario y Representación

Un número binario de 4 bits puede representar valores del 0 al 15. Cada bit tiene un peso:

- Bit 0 (LSB) en GPIO2: peso 1
- Bit 1 en GPIO3: peso 2
- Bit 2 en GPIO4: peso 4
- Bit 3 (MSB) en GPIO5: peso 8

Ejemplo: 1101 binario = $8 + 4 + 0 + 1 = 13$

2.2. Función `gpio_set_pulls()`

Esta función configura las resistencias internas del pin GPIO:

- `gpio_set_pulls(pin, false, true)`: Configura pull-down (estado normal = 0)
- `gpio_set_pulls(pin, true, false)`: Configura pull-up (estado normal = 1)
- `gpio_set_pulls(pin, false, false)`: Pin flotante (no recomendado)

2.3. Operaciones de Bits utilizadas

- valor & 0x01: Máscara para obtener el bit menos significativo
- (valor 1) & 0x01: Desplazar 1 bit a la derecha, luego enmascarar
- (valor 2) & 0x01: Desplazar 2 bits a la derecha, luego enmascarar
- (contador + 1) & 0x0F: Incrementar y asegurar resultado de 4 bits (0-15)

3. Desarrollo

3.1. CMakeLists.txt

Listing 1: Configuración de CMake

```
1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Practica_7)
7
8 pico_sdk_init()
9
10 add_executable(Practica_7
11     Practica_7.c
12 )
13
14 target_link_libraries(Practica_7
15     pico_stdlib
16     hardware_gpio
17 )
18
19 pico_enable_stdio_usb(Practica_7 1)
20 pico_enable_stdio_uart(Practica_7 0)
21
22 pico_add_extra_outputs(Practica_7)
```

3.2. Practica_7.c

Listing 2: Código completo del contador binario

```
1 /*
2  * Practica 7: Contador binario ascendente de 4 bits
3  */
4
5 #include <stdio.h>
6 #include "pico/stdlib.h"
7 #include "hardware/gpio.h"
8
9 #define BIT0 2
10 #define BIT1 3
11 #define BIT2 4
12 #define BIT3 5
13 #define BUTTON 6
```

```
14
15 uint8_t contador = 0;
16
17 void actualizar_gpio(uint8_t valor) {
18     gpio_put(BIT0, valor & 0x01);
19     gpio_put(BIT1, (valor >> 1) & 0x01);
20     gpio_put(BIT2, (valor >> 2) & 0x01);
21     gpio_put(BIT3, (valor >> 3) & 0x01);
22 }
23
24 int main() {
25     stdio_init_all();
26
27     printf("Practica 7: Contador binario\n");
28
29     gpio_init(BIT0);
30     gpio_init(BIT1);
31     gpio_init(BIT2);
32     gpio_init(BIT3);
33
34     gpio_set_dir(BIT0, GPIO_OUT);
35     gpio_set_dir(BIT1, GPIO_OUT);
36     gpio_set_dir(BIT2, GPIO_OUT);
37     gpio_set_dir(BIT3, GPIO_OUT);
38
39     gpio_init(BUTTON);
40     gpio_set_dir(BUTTON, GPIO_IN);
41     gpio_set_pulls(BUTTON, false, true);
42
43     actualizar_gpio(contador);
44
45     while (1) {
46         if (gpio_get(BUTTON) == 1) {
47             sleep_ms(50);
48             while (gpio_get(BUTTON) == 1);
49             sleep_ms(50);
50             contador = (contador + 1) & 0x0F;
51             actualizar_gpio(contador);
52         }
53         sleep_ms(10);
54     }
55     return 0;
56 }
```

4. Diagramas del Circuito

4.1. Diagrama Esquemático

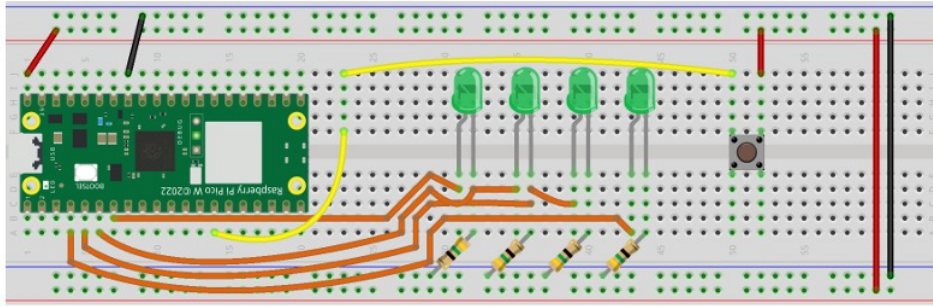


Figura 1: Diagrama esquemático del contador binario de 4 bits

4.2. Diagrama en Protoboard

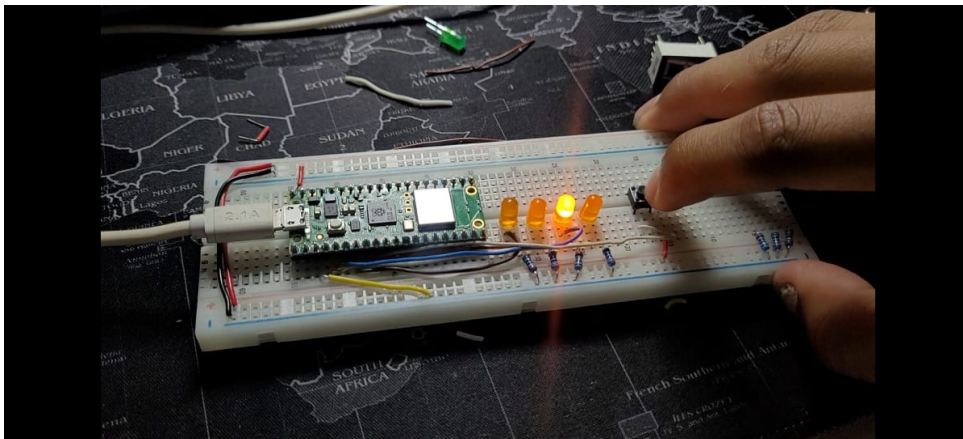


Figura 2: Conexión del contador binario en protoboard

4.3. Conexiones

- BIT0 (LSB - valor 1): GPIO2 (Pin 4) al LED con resistencia 68 Ohm
- BIT1 (valor 2): GPIO3 (Pin 5) al LED con resistencia 68 Ohm
- BIT2 (valor 4): GPIO4 (Pin 6) al LED con resistencia 68 Ohm
- BIT3 (MSB - valor 8): GPIO5 (Pin 7) al LED con resistencia 68 Ohm
- Botón: GPIO6 (Pin 9) a 3.3V (sin resistencia externa, usa pull-down interno)

5. Preguntas de Repaso

5.1. 1. ¿Qué pasa si `gpio_set_pulls(BUTTON, false, true)` cambia a `gpio_set_pulls(BUTTON, true, false)`?

La lógica se invierte. Con pull-up, el estado normal del pin es 1 lógico. Al presionar el botón, se conecta a GND y se lee 0. El programa actual espera un 1 para incrementar, por lo que el contador nunca avanzaría. Para corregirlo, habría que cambiar la condición a `if (gpio_get(BUTTON) == 0)`.

5.2. 2. ¿Cuál es la función del cable rojo que sale del pin 40 al riel positivo de la protoboard?

El pin 40 (VBUS) proporciona 5V provenientes directamente del puerto USB. El cable rojo lleva esta alimentación al riel positivo de la protoboard para alimentar componentes externos como LEDs, sensores, etc. Si se utiliza una fuente externa en la protoboard, el cable puede funcionar en sentido inverso.

5.3. 3. Explica el bloque de código de la línea 14 a 19

La función `actualizar_gpio` recibe un número de 0 a 15 y extrae cada uno de sus 4 bits para encender o apagar los LEDs correspondientes. Por ejemplo, el número 13 en binario es 1101. La función enciende BIT0 (valor 1), apaga BIT1 (valor 2), enciende BIT2 (valor 4) y enciende BIT3 (valor 8), representando $8+4+0+1 = 13$.

6. Conclusiones

- Se implementó exitosamente un contador binario ascendente de 4 bits utilizando 4 LEDs como salida
- Se utilizó la resistencia pull-down interna mediante `gpio_set_pulls()`, eliminando componentes externos
- Se implementó un sistema anti-rebote con retardos de 50 ms para garantizar una sola lectura por pulsación
- Se practicaron operaciones de bits: desplazamientos (`<`) y máscaras (`&`)

6.1. Posibles Mejoras

- Implementar anti-rebote con temporizadores sin bloquear el programa
- Agregar un botón de reset o decremento
- Usar interrupciones GPIO en lugar de polling continuo

7. Referencias

1. Raspberry Pi Foundation. (2024). Raspberry Pi Pico W Datasheet.
2. Raspberry Pi Foundation. (2024). Pico-series C/C++ SDK Documentation.
3. RP2040 Datasheet. (2024). RP2040: A microcontroller from Raspberry Pi.